CISGUARD 360

Product Requirements Document

Unified CIS Compliance, Cross-Layer Security Intelligence & Proof-of-Hardening

Full-scope, no-AI/ML cybersecurity auditing and controlled hardening platform

SCOPE LOCK	SOURCE REQUIREMENTS	INNOVATION
88 planned features	38 mandatory PS entries + 30 industry extras	20 proposed differentiating features

Version: 1.0 | Date: 29 October 2026 | Document status: Proposed development specification

Working project name only. Product name/trademark and benchmark distribution rights must be reviewed before public release.

PRODUCT PROMISE Audit -> Understand -> Plan -> Simulate -> Harden -> Verify -> Prove -> Monitor. Mandatory CIS compliance assessment must function independently of optional innovation modules.

Document control and reading guide

Field	Value
Project	CISGuard 360 - working name
Source	Hackathon PS: Automated CIS Benchmark Compliance & Hardening Auditor for OS, Firewalls, and Network Devices
Current stage	Proposed engineering specification; no implementation/completion claim
Scope	M01-M38 (mandatory), E01-E30 (important extras), I01-I20 (innovations)
Hard constraints	No runtime AI/ML/LLM; no dependency on paid third-party auditing APIs; deterministic results
Default mode	Read-only audits; mutation allowed only through privileged, approved workflows
Control source	Correct CIS benchmark release/profile plus licensed/authorized rule mapping
Interpretation	Mandatory original PS remains intact; enhancements add to rather than replace it

Table of contents

1 Executive summary and product vision

2 Problem, goals, users and use cases

3 Scope, supported platforms and compliance model

4 Detailed 88-feature requirements register

5 Product workflows and screens

6 Logical architecture and technology stack

7 Configuration collection and policy evaluation

8 Risk graph, optimization and simulation

9 Hardening, verification and evidence

10 Security and privacy requirements

11 Data model and API contracts

12 Non-functional requirements and operational targets

13 Validation strategy and acceptance gates

14 Delivery sequencing, dependencies and risk register

15 Definition of done and references

1. Executive summary and product vision

CISGuard 360 is a centralized, GUI-backed, deterministic security configuration assessment and controlled hardening platform. It evaluates applicable CIS Benchmark requirements against live or explicitly labeled offline configurations for Windows, Linux, specified enterprise firewall vendors, and Cisco routers/switches. It generates verifiable findings and reports with remediation guidance, while offering advanced cross-layer risk analysis, constraint-aware planning, safety verification and evidence-backed post-change assurance.

Product principles

- Full source PS coverage is the baseline. None of M01-M38 may be removed to make space for extras.
- The complete 88-feature inventory is retained as planned scope; no feature is labeled complete without evidence.
- Deterministic means identical versioned inputs and rules yield identical auditable results. No runtime LLM/AI/ML is used.
- Read-only by default. Any hardening requires valid privileges, review, backup/prechecks, explicit approval and post-checks.
- Unknown, unavailable or unsupported evidence is never silently treated as a successful compliance check.
- Vendor and firmware compatibility is explicit; a generic adapter does not imply full support for every vendor variant.

Definition of system value

User question	CISGuard 360 answer
What is wrong?	Precisely identified control failures with vendor and benchmark versions.
Where and why?	Actual evidence, affected assets, contextual risk scenarios, rule explanation.
What should change?	Applicable CLI/API guidance, vetted alternatives and constrained plan.
Will it break a service?	Model-supported what-if analysis + service-safety contracts and confidence/coverage.
Did it work?	Re-audit + real connectivity/service checks; signed evidence-linked proof.
Will it stay fixed?	Historical snapshots, scheduled reassessment, drift and alerts.

2. Problem, goals, users and use cases

Problem

Organizations operate heterogeneous Windows/Linux endpoints and servers, firewalls, routers and switches. Manual CIS auditing is inconsistent and slow. Device-specific configuration formats, benchmark revisions, privileged access and multi-device operational dependencies make assessment, safe correction and evidence collection difficult.

Product goals

- G1. Assess all specified platform families and applicable security control categories from one management experience.
- G2. Maintain versioned, customizable, explainable compliance policy packs.
- G3. Present actionable, exportable and auditable results by asset type, risk/severity and control.
- G4. Connect related misconfigurations into evidence-backed cross-device risk scenarios.
- G5. Minimize hardening disruption via constraint-aware planning and simulation where supported.
- G6. Provide approval-gated remediation, meaningful verification, evidence integrity and ongoing monitoring.

Users and permissions

Persona	Needs	Rights (minimum)
Security administrator	Register assets, schedule audits, manage policies	Manage target inventory; initiate assessment
Auditor / compliance officer	Review benchmark mapping and downloadable findings	Read audits and verify evidence; no configuration writes
Network / OS operator	Plan safe changes and execute approved action	Execute bounded remediation in assigned assets
Change approver	Validate high-risk changes and exception requests	Approve/reject; cannot bypass separation-of-duties policy
Organization owner	Control access, retention and infrastructure scopes	Tenant administration and permission management
Executive / manager	Understand compliance trends and remediation progress	Read aggregate reports only

Primary user stories

- As an auditor, I want to upload or collect configurations and see exactly which CIS controls pass, fail or cannot be assessed.
- As an operator, I want a dry-run plan with required service contracts before I change firewall/OS security settings.
- As a reviewer, I want to approve high-risk changes only after seeing backup, simulation and recovery readiness.
- As a manager, I want asset-group trend reports that distinguish coverage from compliance.
- As a security engineer, I want to know if a firewall issue and an endpoint setting jointly increase a network exposure.

3. Scope, platforms and compliance model

In-scope assets and connectivity

Family	Required platform scope	Primary collection channel
Windows	Windows 11 Enterprise/Standalone; Windows Server 2019	PowerShell/WinRM over HTTPS + native policy/registry evidence
Linux	Ubuntu 20.04 and 22.04 LTS; RHEL 8 and 9	SSH + native tools + OpenSCAP where supported
Firewalls	Fortinet; Cisco; Check Point; Sophos; Palo Alto	Vendor/model/firmware-specific SSH or REST/XML/management APIs
Routers and switches	Cisco	Netmiko/Scrapli/pyATS + compatible Ansible collections

Device family / firmware compatibility matrix is maintained separately for each collector, parser, rule evaluator, simulator and remediation adapter. Coverage is audited by version, not inferred from branding.

Controls and outputs

Mandatory coverage includes management-plane Telnet/HTTP/SSH security; ACLs and firewall rules; sensitive ports; ordered/shadowed rule checks; default-deny assessment; unused ports; port security/MAC binding; DHCP snooping; Dynamic ARP Inspection; VLAN 1 management traffic; Syslog; NTP; and SNMPv3/default community weaknesses. OS controls follow the selected benchmark release and profile, including authentication, authorization, services, host firewall, logging and system hardening as applicable.

All audit results include: asset ID; device version; control ID and source/pack version; selected profile; applicability; rule inputs; expected state; observed state; immutable evidence reference/hash; evaluator version; severity; timestamps; status; remediation reference and any limitation.

Status and scoring rules

Status	Meaning / treatment
PASS	Applicable automated control was assessed and met expected condition.
FAIL	Applicable automated control assessed and did not meet expected condition.
NOT_APPLICABLE	Control irrelevant for asset or agreed context; excluded from denominator.
MANUAL_REVIEW	Additional human evidence needed; excluded from assessed denominator until validated.
ERROR	Attempted assessment failed; never counted as pass.
NOT_ASSESSED	No attempt or missing support/evidence; never counted as pass.
EXCEPTION	Policy-approved time-bound exception; retains original technical finding and is reported separately.

SCORING INVARIANT Assessed compliance = PASS / (PASS + FAIL) x 100 for applicable automated checks; show assessment coverage alongside every score. Severity-weighted organizational scores may supplement, never overwrite, this measure.

4. Detailed 88-feature requirements register

Every row is a required product-scope item. The acceptance-evidence column is a minimum validation example, not a claim of current implementation. IDs stay stable across planning, tickets, tests and release evidence.

4.1 Mandatory PS - M01 to M38 (38 entries)

ID	Feature	Functional requirement	Acceptance evidence
M01	Windows 11	Audit Enterprise and standalone Windows 11 configuration against applicable selected CIS profile.	Run on supported Windows 11 test hosts; validate policy results against known secure/insecure configurations.
M02	Windows Server 2019	Collect and assess applicable server-local security controls and relevant policy state.	Validate on a Windows Server 2019 VM; record evidence for each supported control.
M03	Ubuntu 20.04/22.04	Collect and evaluate Ubuntu-specific system, SSH, account, service, firewall and audit settings.	Run version-specific tests on both LTS versions; no unsupported checks silently pass.
M04	RHEL 8/9	Audit RHEL-specific controls including SELinux, audit rules, accounts and services.	Pass independently reviewed tests on both major versions.
M05	Fortinet firewall	Collect and assess supported FortiOS management and policy configuration.	Verify vendor/model/firmware handshake and known fixture results.
M06	Cisco firewall	Support documented Cisco ASA and/or FTD-family flows with explicit family handling.	Execute family-specific collection and tests; label compatibility precisely.
M07	Check Point firewall	Collect management-policy and device configuration from authorized management interfaces.	Validate policy extraction, normalization and checks on supported versions.
M08	Sophos firewall	Collect and parse supported Sophos firewall configuration using applicable API/CLI.	Validate version-specific fixtures and at least one authorized lab integration.
M09	Palo Alto firewall	Collect PAN-OS configuration and policy via supported REST/XML interfaces.	Verify current committed state and relevant objects in findings.
M10	Cisco router	Audit IOS/IOS-XE or other explicitly supported Cisco router families.	Test Cisco router fixtures, command authorization and rule results.
M11	Cisco switch	Audit supported managed Cisco switch families and Layer-2 hardening.	Test VLAN, interface, port-security and switch-specific findings.
M12	Live collection	Securely retrieve actual configuration/state from authorized managed devices.	Audit run logs source, timestamp, identity, retrieval success and evidence hash.
M13	Central multi-target audits	Schedule and execute audits across multiple asset categories from one interface.	Single job contains mixed supported devices; per-device result and failure isolation.
M14	OS collectors	Collect Windows/Linux local policies, users, processes and system security state.	Collector outputs canonical evidence with explicit missing-data status.
M15	Network collectors	Collect firewall, router and switch running/effective configuration as available.	Adapter tags vendor, device family, firmware and collection method.
M16	Deterministic rule engine	Evaluate versioned CIS-aligned controls with reproducible operators, not AI/ML.	Repeat identical input/profile -> identical result and evidence links.
M17	Modular structured rule files	Support maintainable, validated YAML/JSON and XML/SCAP where applicable.	Schema-invalid rule rejected with clear diagnostic; valid pack loads.
M18	Custom baselines	Allow organization thresholds/overrides separate from benchmark source.	Changing override changes only appropriate checks and preserves history.
M19	Benchmark update maintenance	Track benchmark version/profile and controlled change to rule packs.	Old audit remains reproducible; new version diff is reviewable.
M20	Management plane security	Audit Telnet/HTTP management restrictions as applicable to device/config.	Unsafe management interface produces documented failure and evidence.

ID	Feature	Functional requirement	Acceptance evidence
M21	SSHv2, key auth, timeout	Check SSH version, permitted auth method and idle timeout where supported.	Fixtures distinguish compliant/weak/unknown management configurations.
M22	Over-permissive ACLs	Detect broad allows such as permit ip any any in effective context.	Parser respects ACL order/context and produces evidence/location.
M23	Sensitive port exposure	Detect risky inbound rules from broad sources to 22/3389/445 and extensible port list.	Exposure scenario uses actual effective policy and source/destination scope.
M24	Shadowed rules	Identify fully shadowed or unreachable ordered firewall/ACL rules where model supports it.	Constructed fixture detects known shadowed rule without false pass.
M25	Default-deny policy	Assess applicable implicit/explicit final action and exceptions.	Correct results on deny, permit and unknown cases.
M26	Unused switch ports	Check designated unused ports are administratively disabled.	Unassigned enabled ports flagged with inventory-based applicability.
M27	Port security / MAC binding	Verify switch edge-port restrictions and configured limits.	Known secure/insecure port security fixtures return expected outcomes.
M28	DHCP snooping	Evaluate feature state, trusted ports and VLAN applicability.	Flags missing scope or unsafe trust with exact interface evidence.
M29	Dynamic ARP inspection	Evaluate DAI/ARP-inspection settings and trust boundaries.	Flags disabled/mis-scoped protections where supported.
M30	VLAN 1 management traffic	Detect use of default VLAN for management where applicable.	Report actual management VLAN and evidence.
M31	Syslog forwarding	Verify remote logging destinations and security-relevant configuration.	Missing/invalid forwarding recorded as failing or unknown with reason.
M32	NTP synchronization	Verify time source configuration and available sync status.	NTP configured vs merely present distinguished in results.
M33	SNMP security	Flag default v1/v2c strings; validate version 3 controls where supported.	Sample public/private communities fail; configured v3 is inspected.
M34	Central GUI dashboard	Provide searchable device overview, audit initiation and findings pages.	Authorized reviewer navigates complete audit without command line.
M35	Compliance/severity breakdown	Display overall and category compliance with Critical/High/Medium/Low distribution.	Score excludes unassessed/errors; coverage reported separately.
M36	Expected vs actual findings	Present rule, expected value, actual observation and evidence side-by-side.	Every failed automated control links to evidence and rule version.
M37	Actionable remediation	Show vendor/version-specific safe guidance and exact CLI where reliably applicable.	Remediation includes prerequisites, risk warning and command/source mapping.
M38	Reports and scale	Export PDF/HTML/CSV/JSON with passes, fails, severity, evidence and remediation; scale across assets.	Four formats reconcile with same audit; load test records latency and failures.

4.2 Industry-standard enhancements - E01 to E30 (30 entries)

ID	Feature	Functional requirement	Acceptance evidence
E01	Asset inventory	Searchable centralized asset registry.	Create, edit, retire and filter asset records.
E02	Authorized discovery	Discover eligible targets only within approved ranges.	Discovery logs scope; out-of-scope assets are excluded.
E03	Tag/group assets	Group by site, environment, owner and criticality.	Tags control filters and policy assignment.
E04	Scheduled scans	Run recurring audits without manual action.	Schedule triggers expected job and records time.

ID	Feature	Functional requirement	Acceptance evidence
E05	Drift alerts	Detect relevant config changes between snapshots.	Controlled change produces accurate change event.
E06	Configuration snapshots	Retain encrypted, versioned configuration evidence.	Snapshot can be decrypted by authorized actor and hash checked.
E07	Compliance trends	Compare audit scores with consistent benchmarks and coverage.	Time-series view reflects historical results accurately.
E08	Selective audits	Scan chosen devices, groups, rule packs or controls.	Selection matches executed checks and job manifest.
E09	Distributed workers	Execute collectors across approved networks/sites.	Multiple workers isolate job failures and maintain tenant boundaries.
E10	Offline config assessment	Audit uploaded/sanitized device configuration snapshots.	Offline results clearly marked as non-live.
E11	CIS profile selection	Choose appropriate Level 1/Level 2 profiles where available.	Profile changes applicable controls and report label.
E12	Manual-review workflow	Assign controls requiring human validation.	Manual item has reviewer, evidence and auditable verdict.
E13	Risk ranking	Prioritize by severity, exposure, asset criticality and evidence confidence.	Risk output reproducible, factors shown.
E14	Policy exceptions	Time-box exceptions with approver, reason and expiration.	Expired exceptions return to active finding workflow.
E15	Role-based access control	Restrict audit/remediation/report permissions by role and tenant.	Negative authorization tests pass for all privileged actions.
E16	Multi-factor authentication	Protect privileged dashboard accounts with MFA.	MFA-required role cannot perform protected action without MFA.
E17	Secrets vault	Store device credentials/keys outside source code and audit tables.	Application references secret ID only; no plaintext in logs.
E18	Read-only audit mode	Collection never mutates device settings by default.	Audit-mode tests show zero configuration-write calls.
E19	Application audit trail	Record logins, policy changes, approvals, exports and commands.	Event sequence attributable and integrity-checkable.
E20	Policy version history	Diff and restore organization rule pack versions.	Historical audit remains linked to original policy.
E21	Approved auto-remediation	Execute allowlisted supported changes after authorization.	Only compatible, approved actions execute and are journaled.
E22	Dry-run preview	Show intended configuration changes before execution.	Preview contains diff/prerequisites; unsupported preview labeled.
E23	Approval workflow	Require explicit reviewer approval for sensitive changes.	Unapproved action cannot run; approval actor logged.
E24	Backup/recovery	Take backups and provide rollback/compensation for supported actions.	Supported rollback passes test; unsupported clearly escalates.
E25	Post-fix re-audit	Automatically re-evaluate controls after change.	Result linked to original finding and change.
E26	Alerts/webhooks	Send notices for critical events and changed statuses.	Deliver or retry notification; secret values never included.
E27	Ticket integrations	Export/route findings into external issue workflows.	Idempotent ticket creation/update with links.
E28	Executive reporting	Provide concise management-level audit summary.	Summary totals reconcile with technical report.

ID	Feature	Functional requirement	Acceptance evidence
E29	Search/filter interface	Filter by asset, severity, benchmark, status and owner.	Saved filters return correct permitted records.
E30	External integration API	Expose secured machine-readable audit/export interfaces.	Documented versioned API with auth and rate controls.

4.3 Proposed differentiating innovations - I01 to I20 (20 entries)

ID	Feature	Functional requirement	Acceptance evidence
I01	Infrastructure relationship graph	Link assets, zones, interfaces, subnets, services and controls.	Graph edges backed by configuration/observed evidence.
I02	Cross-device correlation	Join related findings into contextual scenarios.	Correlated scenario retains all source finding IDs.
I03	Config-based path analysis	Estimate permitted access paths from effective ACL/NAT/route/host policy.	Known reachable/unreachable test topologies pass.
I04	Root-cause reasoning	Group findings sharing a verifiable underlying cause.	Explanation names cause rule and supporting data.
I05	Evidence-backed risk explanation	Explain every detected cross-layer security scenario.	User can trace each risk to exact normalized facts.
I06	Minimal-change optimization	Select feasible remediation actions minimizing weighted change impact.	Test solver solution obeys constraints and compares alternatives.
I07	Alternative fix comparison	Present viable candidates with impact and trade-offs.	At least two vetted alternatives on supported scenario.
I08	Cross-vendor planning	Combine fixes for related devices in a coordinated plan.	Plan connects vendor-specific actions and dependencies.
I09	Remediation conflict checks	Identify policy, access and resource conflicts before execution.	Known unsafe conflicting plan blocked.
I10	Change sequencing	Order approved actions to protect management access and dependencies.	Ordering respects graph constraints in fixture.
I11	Cross-layer what-if	Simulate selected network/host policy modifications within model coverage.	Compare baseline vs candidate on labeled test topology.
I12	Service-safety contracts	Define traffic/services that must remain accessible.	A violating proposed change is flagged.
I13	Blast-radius estimation	Identify endpoints/services potentially impacted by a change.	Affected-set reconciles with known connectivity fixture.
I14	Safety-gated execution	Prevent execution when required safety tests fail or unknown policy blocks.	Failing/unverified mandatory contract prevents automated run.
I15	Staged rollout	Apply supported changes through controlled batches and gates.	Unhealthy first batch prevents later execution.
I16	Dual verification	Recheck CIS configuration and real service/connectivity behavior.	Success requires both requested check types to pass.
I17	Residual risk measurement	Report remaining risk/exposure following verified hardening.	Post-state calculation reproducible and evidence-linked.
I18	Proof-of-Hardening dossier	Package before/after evidence, approvals, actions, checks and signatures.	Third-party verifier checks hashes/signatures and event chain.
I19	Benchmark upgrade preview	Compare applicability/evaluation changes across two policy versions.	Preview shows added/removed/modified rules and affected devices.
I20	Explainable coverage decisions	Show rule reasoning, unsupported semantics, errors and evidence coverage.	No unsupported/unknown state appears as PASS; reason accessible.

5. User workflows and screen specification

5.1 End-to-end sequence

Step	User/system action	Required guard or output
01 Discover	Register target and consent/scope	Inventory identity, owner and authorization
02 Collect	Read live or import offline configuration	Collector provenance, version, signed/hash evidence
03 Audit	Choose policy release/profile; evaluate rules	Pass/Fail/NA/Error/Manual outcomes
04 Understand	Build contextual risk graph	Evidence-linked scenario with confidence/coverage
05 Plan	Generate vetted candidate actions	Constraints, alternatives, impact and prerequisites
06 Simulate	Compare baseline/proposed models	Safety-contract verdict + modeling limitations
07 Approve	Review risk/backup/permissions	Explicit authorized approver decision
08 Harden	Execute approved vendor-specific actions	Execution log, checkpoints, stop gates
09 Verify	Recollect configuration and test services	Both security control and service checks
10 Prove/Monitor	Export dossier; schedule drift audits	Report, evidence integrity, alerts

5.2 Required web pages

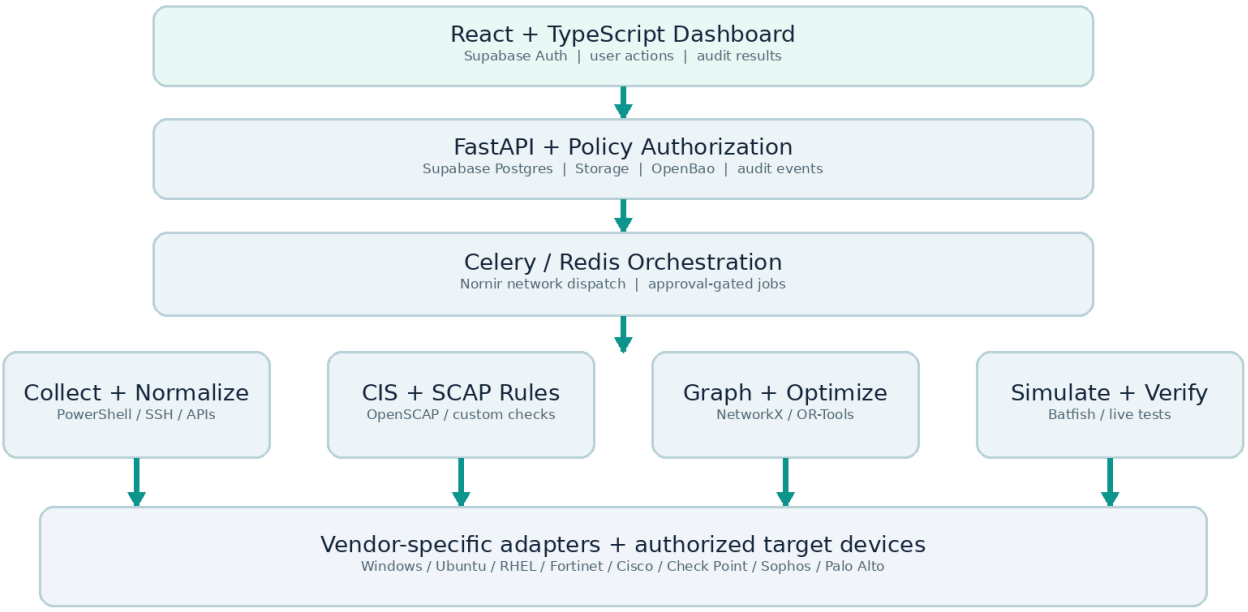
Page	Core functions
Login / Organizations	Supabase Auth, MFA, membership, tenant switching
Overview dashboard	Global/asset compliance score, coverage, severity, job status and trend
Asset inventory	Devices, vendor/version, site/groups, credentials reference and last seen
Policy library	Benchmark release/profile, mappings, overrides, exception and manual checks
Audit launcher / queue	Target selection, read-only scans, schedule, per-target status and retries
Finding detail	Expected vs actual, source rule/evidence, severity and remediation
Infrastructure graph	Cross-device risk links and explainable security scenarios
Planner / simulation	Candidate remediations, conflict matrix, service contracts and preview
Approval / hardening	Approvals, prechecks, execution, progress, recovery readiness
Verification	Post-state control/service tests, residual risk and proof dossier
Reports / history	PDF/HTML/CSV/JSON export, snapshots and drift timeline
Administration	Users, RBAC, secrets references, integrations, retention and audit trail

5.3 Interface safety and correctness

- Always show asset and benchmark profile/version on every finding and report.

- For any risky operation, UI must show a human-readable diff, loss-of-access warning and explicit confirmation.
- Run Audit (read-only) and Apply Remediation (mutation) must be separate permissions and actions.
- Any incomplete parser/model state must display UNKNOWN/NOT_ASSESSED instead of fabricated PASS.
- A failed/blocked action must show a meaningful recovery path and preserve previous evidence.

6. Architecture and complete technology stack



Evidence flows into reports, drift history and signed Proof-of-Hardening packages.

Logical architecture; strict control boundaries run through FastAPI and privileged background workers. Persistent datastore operations and storage authorization are tenant-aware.

Layer	Selected tools / libraries	Responsibility
Frontend	React, TypeScript, Vite, Tailwind, shadcn/ui, Recharts, TanStack Query/Table, React Hook Form, Zod, Cytoscape.js, React Flow	Dashboard, policy forms, evidence and graph UI
API	Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy, Psycopg, Alembic, HTTPX	Validation, authorization, APIs, audit coordination
Identity/data	Supabase PostgreSQL, Auth/MFA, RLS, Storage, Realtime, CLI/Studio	Multitenancy, metadata, evidence, session support
Workers	Celery, Redis, Celery Beat, Nornir	Scan/hardening queue, site-specific workers, schedule
OS audit	PowerShell, WinRM, pywinrm/pypsrp, OpenSCAP, ComplianceAsCode, osquery, Microsoft SCT/LGPO	Native OS evidence and compatible SCAP checks
Network access	Netmiko, Scrapli, Paramiko, TextFSM, TTP, ciscoconfparse2, pyATS/Genie, vendor REST/XML	Device collection and vendor normalization
CIS policy	Custom Python evaluator, PyYAML, JSON Schema, XML/XCCDF/OVAL, OPA for policy constraints	Reproducible controls, pack updates, exceptions
Intelligence	NetworkX and authored graph/correlation algorithms	Relationship and exposure reasoning
Planning/ simulation	OR-Tools CP-SAT, Batfish, Pybatfish, custom vendor behavior models	Constraint-aware plans, topology and traffic analysis
Execution	Ansible Core, Ansible Runner, compatible vendor collections, PowerShell, APIs	Approval-gated changes, verification and recovery
Secrets/evidence	OpenBao, Cryptography, Jinja2, WeasyPrint, SHA-256	Secrets, encrypted artifacts, signed evidence, reports

Layer	Selected tools / libraries	Responsibility
Quality/ops	Docker Compose, Linux, GitHub Actions, Pytest, Hypothesis, Playwright, Semgrep, Bandit, Trivy, Prometheus, Grafana, Loki, OpenTelemetry	Testing, deployment, monitoring

6.1 Architecture rules

- Supabase hosted or self-hosted is supported by stable PostgreSQL/Auth/Storage interfaces; sensitive enterprise deployments should have controlled residency.
- OpenBao stores device credentials and API tokens; database contains references rather than secret values.
- Frontend never receives privileged keys or direct device-access capability.
- Long-running simulations and device operations run asynchronously in isolated workers with bounded privileges.
- OpenSCAP/ComplianceAsCode are supporting engines; custom evaluation covers missing CIS-specific controls or platforms.
- Batfish does not automatically support every firmware/vendor feature: capability-aware custom modeling and real verification are mandatory.

7. Collector, normalization and policy engine specification

7.1 Device adapter contract

Each adapter implements: probe_version(), collect(read_only=True), parse(), normalize(), supported_controls(), validate_credentials(), get_remediation_capabilities(), and an optional execute_approved_change(). Adapters must explicitly enumerate unsupported versions/capabilities and ensure read-only probes do not modify state.

Normalized object	Required fields
Asset	asset_id, org_id, family, vendor, model, os/firmware_version, collector_version
Management	interfaces, enabled protocols, auth settings, timeout and exposed subnet metadata
Network policy	rules, order, zones, address/service objects, source, destination, action, NAT where known
OS security facts	users/groups, service states, auth/audit policies, firewall and file/registry configuration
Topology	interfaces, links, subnets, routes, VLANs, trusted/untrusted scope
Evidence	timestamp, collection method, command/API endpoint ID, raw protected artifact URI, integrity hash

7.2 Sample rule schema (conceptual)

```
id: org-ssh-telnet-001
source: organizational-override
applies_to: [cisco_ios]
when:
  fact: management.telnet_enabled
  operator: equals
  value: true
expected:
  management.telnet_enabled: false
severity: high
remediation_template: cisco_ios_vty_ssh_only
requires_manual_review: false
```

This is an illustrative organizational override, not a claim of exact official CIS control wording. Official rule IDs and requirements must be bound to an authorized benchmark release and reviewed.

7.3 Deterministic rule execution

- Validate pack schema and signature; resolve platform, benchmark edition/profile, organization overrides and exceptions.
- Evaluate applicability separately from pass/fail; missing evidence produces ERROR or NOT_ASSESSED.
- Run allowlisted operators: equals, contains, regex with limits, range, set membership, CIDR containment, ordered ACL semantics, optional XCCDF/OVAL evaluation.
- Persist expected, actual, raw evidence reference, normalized fact, evaluator version, confidence/coverage limitations and remediation pointer.
- Produce scoring and export records from one canonical results store, never from separate front-end calculations.

7.4 Benchmark rights and mappings

CIS benchmark contents are subject to terms. Do not redistribute proprietary benchmark text or machine-readable packs without rights. Store source attribution, version, profile, control mapping and rights metadata. Organizational-authored rules must be labeled separately from official benchmark checks. OpenSCAP/ComplianceAsCode content must not be misrepresented as a complete CIS benchmark mapping without validation.

8. Security intelligence, optimization and simulation

8.1 Cross-layer evidence graph

Graph nodes represent assets, interfaces, networks, routes, services, policies, security controls and findings. Typed, evidence-linked edges encode hosted-on, attached-to, routes-to, allows/denies, depends-on and violates. Evidence quality and topology completeness are explicit attributes.

8.2 Risk correlation

Author deterministic scenario definitions: required predicates, source-finding IDs, reachable path conditions, exclusions and confidence/unknown handling. The system must not assert a validated attack path solely from textual similarity of two findings.

8.3 Minimal-change remediation planner

Use a vetted catalog of vendor/version-specific candidate actions. OR-Tools CP-SAT selects actions to satisfy security targets while minimizing weighted disruption/change cost subject to immutable service-safety and access-management constraints. If no feasible plan exists, return UNSAT with violated constraints; do not fabricate a recommendation.

Constraint type	Example
Security	Eliminate prohibited public SSH access under effective policy.
Availability	Preserve admin subnet to managed device access path.
Dependencies	Apply route/ACL updates in an order that avoids lockout.
Capability	Do not choose API action unsupported by running firmware.
Approval	Execution needs approver + operator separation for critical operations.
Recovery	Prohibit automatic action without verified recoverability where policy requires it.

8.4 What-if model and confidence

Construct baseline and candidate snapshots. Analyze supported routing, ACL, NAT and host/network policy behavior, then test service-safety contracts. Prediction output: SAFE_IN_MODEL, VIOLATES_CONTRACT, UNSUPPORTED, or INSUFFICIENT_EVIDENCE; never present model prediction as universal proof of operational safety.

8.5 Modeling gaps

Batfish is the initial supported-vendor modeling engine; custom semantics adapters are required for unsupported platforms or behaviors. For unsupported cases, design conservative prechecks, staged change rollout and measured post-change tests. Unknown model coverage should block fully autonomous hardening whenever a mandatory contract depends on it.

9. Controlled hardening, verification and proof

9.1 State machine

DRAFT -> SIMULATED -> PRECHECKED -> PENDING_APPROVAL -> APPROVED -> EXECUTING -> VERIFYING -> VERIFIED / FAILED / ROLLED_BACK / NEEDS_MANUAL_RECOVERY. Terminal states are immutable audit events. Reattempts create linked executions rather than overwriting evidence.

9.2 Safety gates

- Scope authorization and exact device identity/version verified.
- Credentials retrieved only by eligible worker through OpenBao; no logs of secrets.

- Validated command template or API action bound to firmware and expected state.
- Configuration backup and explicit recovery plan available, or action requires elevated manual workflow.
- Dry-run or pre-change simulation performed where supported; all mandatory service-safety contracts satisfied.
- Explicit human approval captured; approval invalidated if plan or target configuration changed.
- Rate limiting, timeout, idempotency, partial failure handling and post-change connectivity verification enforced.

9.3 Verification requirements

Automatic success requires both relevant compliance control pass and requested service checks pass. If only configuration passes but service checks fail/unknown, state remains failed or not fully verified. Changes that cannot be reversed must be identified ahead of time with manual recovery requirements.

9.4 Proof-of-Hardening package

Produce an evidence dossier referencing original failure, source artifact hash, benchmark release, approved change plan, simulation contract outputs, operator/approver identities, execution log, new configuration hash, re-audit outcomes, post-service tests, residual risk and cryptographic signature/integrity manifest.

10. Product security and privacy

Security requirements

ID	Control	Requirement
SEC-01	Tenant isolation	RLS for client-accessible data and tenant-aware backend enforcement on all routes and worker tasks.
SEC-02	Authentication	Supabase Auth; MFA for privileged roles; validate JWT issuer, audience, signature, expiration.
SEC-03	Authorization	Granular permissions: view, scan, policy edit, approve, remediate, export and admin.
SEC-04	Secrets	OpenBao for privileged device keys/tokens; zero plaintext in DB, logs, reports, crash dumps.
SEC-05	Transport	TLS to backend and vendors; SSH host key checking; reject unsafe verification bypass by default.
SEC-06	Collection	Read-only by default, least privilege, explicit scope and tenant ownership.
SEC-07	Evidence	Encrypt sensitive artifacts, hash, sign manifests and restrict access/retention.
SEC-08	Execution	Allowlisted commands/APIs, change approvals, independent pre/post checks, full activity record.
SEC-09	Software supply chain	Pinned dependencies, CI secret scanning, SAST, container scanning and patches.
SEC-10	Availability	Worker isolation, retry with idempotency, task timeouts and recoverable queue state.
SEC-11	Privacy	Data minimization, retention controls, no sensitive config sent to external LLMs.
SEC-12	Audit integrity	Evidence signature validation and restricted append-only event access.

Threats to consider

- Compromised operator account triggers unauthorized device change. Mitigation: MFA, RBAC, separation of duties and mandatory approval.
- Malicious configuration content exploits parser. Mitigation: input limits, safe parsing, sandboxed/scoped workers and no eval().
- Cross-tenant evidence leakage through object URLs. Mitigation: private storage, scoped signed URLs and backend checks.
- Secret exposure in configuration snapshots and CLI logs. Mitigation: encrypted storage, redaction, limited retention.
- Network lockout after remediation. Mitigation: precheck, access-preserving contracts, staged rollout and verified recoverability.
- Discrepancy between modeled and real behavior. Mitigation: capability bounds, real post-change service checks and explicit unsupported states.

11. Data model and API contracts

11.1 Core tables and storage

Domain	Tables / objects
Identity / inventory	organizations, organization_members, assets, asset_groups, credential_references
Policies	benchmark_packs, policy_rules, policy_overrides, policy_exceptions, rule_versions
Audits	audit_jobs, audit_results, findings, manual_reviews, config_snapshots
Intelligence	security_relationships, risk_scenarios, asset_dependencies
Changes	remediation_plans, remediation_actions, simulation_runs, safety_contracts, approvals
Evidence and reporting	verification_runs, audit_evidence, reports, audit_events
Operations	notifications, integrations, schedules, worker_heartbeats

Supabase private buckets: audit-evidence, config-snapshots, device-backups, generated-reports. File data encrypted before upload where sensitivity warrants; DB metadata references content hashes and access scope.

11.2 Representative versioned REST API

Method / route	Description / rule
POST /api/v1/assets	Register device; owner, platform and authorization required.
GET /api/v1/assets	List tenant-scoped inventory.
POST /api/v1/audits	Create read-only multi-target audit.
GET /api/v1/audits/{id}	Status, per-asset progress, failures and coverage.
GET /api/v1/findings	Filter by audit, severity, control and asset.
GET /api/v1/findings/{id}	Evidence, expectations, remediation and policy references.
GET /api/v1/policies	Available authorized benchmark packs and overrides.
POST /api/v1/simulations	Create what-if job using versioned snapshots.
POST /api/v1/plans	Generate vetted remediation plan.
POST /api/v1/plans/{id}/approve	Submit authorized approval bound to plan hash.
POST /api/v1/plans/{id}/execute	Start privileged approved execution; safety gates required.
GET /api/v1/verification/{id}	Read compliance + service test outcome.
POST /api/v1/reports	Generate PDF/HTML/CSV/JSON and proof export.
GET /api/v1/audit-events	Privileged access to immutable-oriented event record.

11.3 Event types

audit.requested, audit.started, evidence.collected, control.evaluated, finding.created, scenario.correlated, plan.generated, simulation.completed, approval.granted/rejected, action.started/completed/failed, verification.completed, evidence.signed, drift.detected, report.generated. Every event carries organization scope, correlation ID, event time and actor/system identity.

12. Non-functional requirements

The values below are engineering acceptance targets to benchmark, not claims of achieved performance. Test environments, data scale and vendor characteristics must accompany reported measurements.

Area	Target / design expectation	Verification
Determinism	Identical normalized facts + policy version => identical verdict.	Repeatability tests across test vectors.
Reliability	Per-device failure does not abort other devices.	Fault injection with unreachable/malformed targets.
Scalability	Architecture supports 1,000 registered assets; audit throughput measured by vendor and worker pool.	Load tests with concurrency and rate limits.
Security	Zero direct browser-to-device privileged access; tenant isolation enforced.	Authorization penetration and negative tests.
Audit coverage	Reports distinguish assessed from unassessed controls by platform/profile.	Reconciliation against rule inventory.
Performance	Dashboard p95 API < 1 s for ordinary paginated reads in benchmark environment; long jobs async.	Measured API tests under specified load.

Area	Target / design expectation	Verification
Recovery	Failed jobs retry safely; action recovery paths tested where supported.	Worker kill, timeout, partial change tests.
Retention	Config/evidence retention customizable with secure deletion policy.	Retention and access-policy tests.
Interoperability	REST, vendor adapters, standard exports, optional SCAP import.	Contract and integration testing.
Observability	Structured logs, traces, metrics without secret leakage.	Log review and monitoring dashboards.
Accessibility	Keyboard navigation and readable color contrast on key views.	Accessibility audit of core pages.
Maintainability	Versioned adapters/rules and documented compatibility matrix.	CI schema and regression tests.

13. Validation strategy and acceptance gates

13.1 Test matrix by layer

Scope	Test categories	Required evidence
OS matrix	Windows 11, Server 2019, Ubuntu 20.04/22.04, RHEL 8/9	Known secure/insecure fixture vectors + VM validation.
Firewall vendors	Fortinet, Cisco, Check Point, Sophos, Palo Alto	Firmware/version, parser fixture, API/CLI test and expected control outcomes.
Cisco network	Router + managed switch families	ACL semantics, VLAN, ports, SNMP, DHCP snooping, DAI fixtures.
Rule engine	Schema, applicability, operators, exception, upgrade	Repeatability, property tests, manual review, error states.
Simulation	Supported policy/NAT/routing changes and unknown features	Prediction vs lab configuration/traffic results.
Planning	Feasible, infeasible and conflicting constraints	Solver output validity and plan comparison.
Hardening	Dry-run, approval, backup, execution, recovery, permissions	Authorized change journal + negative safety tests.
Reporting	PDF/HTML/CSV/JSON and signed dossier	Numbers and details reconcile to source results.
Operational security	MFA, RLS, RBAC, secrets, logs and storage	Security and isolation test evidence.
Scale and failure	Concurrent audit workers, offline endpoints, timeouts	Throughput, failure isolation and queue recovery metrics.

13.2 Per-feature done criteria

- A feature has a versioned specification, owner, test cases, and correct real output for each declared supported configuration.
- Collector or engine behavior is verified on fixtures and authorized OS/device lab conditions when applicable.
- Known failure/unknown/unsupported paths are tested, documented and visible in the GUI and exports.
- Security threat and privilege impact reviewed; no credentials or sensitive config leak.
- Evidence (logs, snapshots, screenshots, test output) attached and linked to the stable feature ID.
- No feature is marked complete merely because a UI control exists or a library was installed.

13.3 Acceptance gate summary

RELEASE GATE The planned product can claim 88/88 features working only after every requirement has passed its platform-specific acceptance tests, appropriate security testing, and report/evidence reconciliation. Hardware or vendor firmware not tested must be explicitly disclosed as unverified.

14. Delivery sequence, dependencies and risks

Phases define engineering build order and test gates, not removal or postponement of any requirement from final scope. A phase can proceed in parallel when cross-team interfaces are stable.

Phase	Outputs and acceptance gate
P0 - Source and access	Benchmark/legal source review; vendor/test-lab inventory; 88-ID requirements and ownership frozen.
P1 - Foundation	Supabase Auth/RLS, FastAPI, worker queue, OpenBao, audit event model, baseline UI.
P2 - Collectors	Windows/Linux + required firewall/Cisco adapter families, canonical schema and fixture suite.
P3 - Compliance	Versioned rule engine, CIS mappings, status/coverage, dashboard, remediation and all report formats.
P4 - Operations	Inventory, schedules, snapshots, drift, alerts, RBAC, integrations, offline/distro scans.

Phase	Outputs and acceptance gate
P5 - Intelligence	Cross-layer graph, risk scenarios, root cause, explainable risk/coverage.
P6 - Planning/simulation	OR-Tools plan selection, Batfish/custom policy models, service contracts, blast radius.
P7 - Hardening/proof	Approvals, Ansible/vendor actions, prechecks, recovery, dual verification and evidence dossier.
P8 - Quality and closure	All M/E/I acceptance tests, device matrix review, security/performance audit and handover.

14.1 Key risks and actions

Risk	Consequence	Mitigation / dependency
Vendor appliance/version access	Cannot validate vendor-specific behavior	Authorized hardware/VM or licensed lab + fixtures, publish test coverage.
CIS benchmark rights	Cannot legally ship proprietary source content	License/terms review, original mapping and user-supplied authorized packs.
SCAP content gaps	Some Ubuntu/CIS controls not covered	Custom versioned tests, mapping review, no false-pass treatment.
Simulation incompleteness	False operational safety assumption	Model coverage states, real service checks and conservative approval gates.
Non-reversible changes	Risk of management lockout or outage	Pre-check, rollback where supported, compensating/manual recovery.
Sensitive network data	Credential/config leakage	OpenBao, encryption, least privilege, minimized exports, local hosting.
Scale and time	Large mixed audit fleet exhausts workers	Bounded concurrency, distributed workers, retries and per-vendor rate controls.
Unverified innovation claims	Misleading evaluation or unsafe usage	Publish test evidence and differentiation as integration, not world-first.

15. Final definition of done, traceability and references

Scope traceability

Requirement source	Stable IDs	Count	Status at document creation
Original PS feature register	M01-M38	38	Specified / not verified implemented
Industry-standard enhancements	E01-E30	30	Specified / not verified implemented
Differentiating innovation proposals	I01-I20	20	Specified / not verified implemented
TOTAL	M + E + I	88	All retained in complete target scope

Release definition of done

- All required source PS assets and control categories represented, with versioned compatibility and an explicit tested matrix.
- All 88 stable IDs have acceptance evidence, correct real inputs/outputs and documented error behavior.
- No runtime AI/ML/LLM and no required paid external auditing APIs.
- PDF/HTML/CSV/JSON reports and remediation guidance reconcile with stored audit evidence.
- Privileged hardening is safe by default, authorized, auditable and independently verified.
- Open-source license obligations, CIS benchmark distribution rights and vendor-use terms reviewed.

- Known limitations are documented and not disguised as fully supported behavior.

Reference sources and tooling documentation

CIS Benchmarks - https://www.cisecurity.org/cis-benchmarks

Supabase docs - https://supabase.com/docs

OpenSCAP - https://www.open-scap.org/

ComplianceAsCode / SCAP Security Guide - https://github.com/ComplianceAsCode/content

osquery - https://osquery.io/

Microsoft Security Compliance Toolkit - https://learn.microsoft.com/en-us/windows/security/operating-system-security/device-management/windows-security-configuration-framework/security-compliance-toolkit-10

Batfish - https://batfish.org/

Google OR-Tools - https://developers.google.com/optimization

Ansible docs - https://docs.ansible.com/

Nornir - https://nornir.readthedocs.io/

Cisco pyATS - https://developer.cisco.com/docs/pyats/

OpenBao - https://openbao.org/docs/

These are implementation reference starting points; validate current vendor/device/version support, security advisory status, and license terms during build. The original hackathon statement supplied by the team is the authoritative scope source.

ENGINEERING COMMITMENT Do not silently delete any of the 88 feature IDs. For every blocker, investigate supported tools, custom algorithms/adapters, and truthful compatibility evidence; record any unverified capabilities without falsely claiming them as functioning.